

# DNN for Network Optimisation

Geant Dataset(Anonymised)

Weakly Review

Aug 8th

# Topology of Geant Dataset

- Nodes - 23
- Links -74
- Time period - Data stored after every 15 min.
- Duration - 1<sup>st</sup> jan to 29<sup>th</sup> April
- Dimension of dataset -  $10772 \times 529(23 \times 23)$
- Graph =  $G(V,E)$  ,where V is nodes E is Edges
- Capacity of Edges =  $C(e) = 40000$
- Load on Edges =  $L(e)$



# Parse Traffic matrix from XML file to Dataset

|       |                              |            |             |            |             |            |          |            |           |            |     |           |            |          |
|-------|------------------------------|------------|-------------|------------|-------------|------------|----------|------------|-----------|------------|-----|-----------|------------|----------|
| [7]:  | tn                           |            |             |            |             |            |          |            |           |            |     |           |            |          |
| [7]:  | timestamp                    | 0          | 1           | 2          | 3           | 4          | 5        | 6          | 7         | 8          | ... | 520       | 521        |          |
| 0     | scr                          | 1.0000     | 1.0000      | 1.0000     | 1.0000      | 1.0000     | 1.0000   | 1.0000     | 1.0000    | 1.0000     | ... | 23.0000   | 23.0000    | 23.0     |
| 1     | des                          | 1.0000     | 2.0000      | 3.0000     | 4.0000      | 5.0000     | 6.0000   | 7.0000     | 8.0000    | 9.0000     | ... | 15.0000   | 16.0000    | 17.0     |
| 2     | unique_id                    | 1.0000     | 2.0000      | 3.0000     | 4.0000      | 5.0000     | 6.0000   | 7.0000     | 8.0000    | 9.0000     | ... | 521.0000  | 522.0000   | 523.0    |
| 3     | IntraTM-2005-01-01-00-30.xml | 14946.8978 | 122478.1600 | 19470.0356 | 124992.7200 | 22715.8667 | 192.2044 | 23830.7200 | 6552.9333 | 17807.4578 | ... | 6096.4533 | 9604.3644  | 123151.5 |
| 4     | IntraTM-2005-01-01-00-45.xml | 15535.5111 | 99302.8356  | 13298.2133 | 120400.4533 | 10914.4089 | 47.6711  | 13403.5556 | 5445.9111 | 14987.2622 | ... | 4719.2356 | 8850.0889  | 124111.5 |
| ...   | ...                          | ...        | ...         | ...        | ...         | ...        | ...      | ...        | ...       | ...        | ... | ...       | ...        | ...      |
| 10770 | IntraTM-2005-04-29-15-45.xml | 13133.4133 | 10394.5778  | 184.5333   | 8716.8800   | 3011.9111  | 29.4400  | 3099.1111  | 1008.0356 | 24096.0978 | ... | 1581.2089 | 7103.7067  | 51927.7  |
| 10771 | IntraTM-2005-04-29-16-00.xml | 12643.1556 | 15844.7822  | 4500.2311  | 5164.8533   | 2540.6400  | 9.6533   | 4762.1156  | 1018.6311 | 32094.4622 | ... | 3960.9867 | 7030.0356  | 41533.7  |
| 10772 | IntraTM-2005-04-29-16-15.xml | 10472.4267 | 15138.0444  | 2211.9822  | 6555.9289   | 2079.5111  | 16.9600  | 9332.1511  | 978.7022  | 29688.9778 | ... | 897.5467  | 13772.0800 | 42271.8  |
| 10773 | IntraTM-2005-04-29-16-30.xml | 12748.4267 | 10333.0933  | 5219.4044  | 5422.0356   | 2855.2000  | 23.1022  | 4873.6711  | 721.5289  | 38600.2578 | ... | 1172.8533 | 6608.0267  | 35312.4  |
| 10774 | IntraTM-2005-04-29-16-45.xml | 11079.4489 | 7934.8089   | 8086.1778  | 5333.2000   | 2541.8133  | 57.8578  | 7024.0800  | 899.6089  | 33403.9556 | ... | 2369.5289 | 6619.0844  | 68135.9  |

10775 rows × 531 columns

# Flow generation from a single Traffic Matrix

A flow is between any two nodes.

- For every TM we have assumed 1000 flows to be generated
- Each flow ( $f_i$ ) contain the following information which are calculated from traffic matrix
- Time of arrival( $TOA_i$ ), Source Node( $S_i$ ), Destination node( $D_i$ ), Bandwidth( $B_i$ ), Duration of Flow( $Dur_i$ )
- $f_i = (TOA_i, S_i, D_i, B_i, Dur_i)$

# Procedure of Flow generation

A flow is between any two nodes.

- For a single TM we Iterate over all Pair of S and D
- $TM = TM_{ij}$  where  $i, j \in (1, 23)$
- if  $TM_{ij} > 0$  we calculate number of flows that exist between each pair as  $K_{ij}$ .
- $K_{ij} = 1000 \times \frac{TM_{ij}}{\sum \sum TM_{ij}}$
- The  $K_{ij}$  must be a integer and we round them off. and the actual number of flow is different then 1000 that we assumed.
- Now, for every  $TM_{ij}$  that for exist each pair of S and D, there is  $K_{ij}$  that represent the number of nodes b/w that S and D.

# Procedure of Flow generation

A flow is between any two nodes.

- Time of arrival and Duration are sampled from uniform and Gaussian distribution respectively.
- $TOA_{ij}^k = \mathcal{U}(T_{tm}, T_{tm} + 900)$  in seconds, where  $T_{tm}$  represents the time associated with a particular TM and 900 is time after which another TM is recorded.
- $DUR_{ij}^k = \mathcal{N}(60, 12)$  in seconds
- Now the traffic volume for each flow between any S and D  $Tr_{ij}^K$  is calculated using Gaussian distribution such that  $\sum Tr_{ij}^K = TM_{ij}$ . and the Bandwidth associated to the flow is calculated as ratio of volume by duration.

# Procedure of Flow generation

A flow is between any two nodes.

- $Tr_{ij}^k = \mathcal{N}(\frac{TM_{ij}}{K_{ij}}(mean), \frac{TM_{ij}}{K_{ij} \times 3}(variance))$
- $B_{ij}^k = \frac{Tr_{ij}^k}{DUR_{ij}^k}$
- $f_{ij}^k = (TOA_{ij}^k, S_{ij}^k, D_{ij}^k, B_{ij}^k, Dur_{ij}^k)$  we obtain K flows with following information for each S and D pairs.
- Now for an entire single TM around 1000 flows should be generated with all of these five information.

# Flow for a single TM

Notice here 1357 rows which are more than 1000 as we rounded off  $K_{ij}$  to higher integer. Also TM can have different number of flows.

```
[17]: flow.iloc[:1357,1:]
```

```
[17]:
```

|      | source_node | destination_node | TOA        | Duration  | Bandwidth  |
|------|-------------|------------------|------------|-----------|------------|
| 0    | 1           | 1                | 765.045068 | 64.190580 | 111.696556 |
| 1    | 1           | 2                | 424.349994 | 79.090954 | 185.771548 |
| 2    | 1           | 2                | 712.106180 | 70.963415 | 196.858004 |
| 3    | 1           | 2                | 599.658852 | 74.960517 | 149.926116 |
| 4    | 1           | 2                | 278.131130 | 52.369863 | 187.950481 |
| ...  | ...         | ...              | ...        | ...       | ...        |
| 1352 | 23          | 19               | 465.248465 | 53.790025 | 22.144708  |
| 1353 | 23          | 20               | 407.926617 | 83.197943 | 38.080642  |
| 1354 | 23          | 21               | 2.075432   | 69.243544 | 66.926802  |
| 1355 | 23          | 22               | 820.022896 | 49.373708 | 212.907736 |
| 1356 | 23          | 23               | 281.451265 | 60.300495 | 6.188268   |

1357 rows x 5 columns

Flow generation for four same days of different week. TM are generated with TOA changing according to the recording time of TM.

Here, TOA of last flow is 345000 approx. which when divided by  $24 \times 60 \times 60$  gives us the number of days data have been recorded approx which is 3.99

```
[18]: flow.iloc[:,1:]
```

```
[18]:
```

|        | source_node | destination_node | TOA           | Duration  | Bandwidth  |
|--------|-------------|------------------|---------------|-----------|------------|
| 0      | 1           | 1                | 765.045068    | 64.190580 | 111.696556 |
| 1      | 1           | 2                | 424.349994    | 79.090954 | 185.771548 |
| 2      | 1           | 2                | 712.106180    | 70.963415 | 196.858004 |
| 3      | 1           | 2                | 599.658852    | 74.960517 | 149.926116 |
| 4      | 1           | 2                | 278.131130    | 52.369863 | 187.950481 |
| ...    | ...         | ...              | ...           | ...       | ...        |
| 511377 | 23          | 19               | 344922.001364 | 66.264418 | 12.435164  |
| 511378 | 23          | 20               | 345062.165187 | 57.685365 | 123.382420 |
| 511379 | 23          | 21               | 344709.822412 | 30.630018 | 94.979595  |
| 511380 | 23          | 22               | 345305.290133 | 44.079684 | 81.270798  |
| 511381 | 23          | 23               | 344970.842939 | 49.391602 | 0.988563   |

511382 rows × 5 columns

# Maximum Link utilization Algorithm

DNN will be trained over output of this algorithm

- Utilization vector - This vector keeps track of the current utilization of each link. given by load ( $I(e)$ ) divided by the capacity of that link ( $C(e)$ )
- Python pulp library is used to implement MLU algorithm for the flows generated above.
- We initialize link utilization vector to zero, and generate all simple paths between source and destination nodes for each flow. we then define Constraints to ensure each flow uses a single path, maintain path consistency, and enforce link capacity limits. Then The objective function maximizes total link utilization. And the same is repeated for each flow updating the utilization vector at arrival and departure of flow. We record Utilisation vector before and after each flow event, storing them in a CSV file for analysis.

# Maximum Link utilization Algorithm

## Initialization, Constraints and Objective function.

- **Utilization Vector:** Initialize an array to track the utilization of each link.
- **Optimization Problem:** Set up the problem as a maximization task.
- **Single Path Constraint:** Ensure each flow uses exactly one path.
- **Path Consistency:** Maintain consistency between all the connecting edges.
- **Edges Capacity Constraints:** Ensure that the flow on each edge does not exceed its capacity.
- **Maximize Total Link Utilization (objective):** Define the objective function to maximize the total utilization of all links.

# Maximum Link utilization Algorithm

## Updating and Tracking

- **Function to Get Utilization Vector:** Define a function to compute the current utilization vector.
- **Track Active Flows:** Maintain a list of currently active flows.
- **Flow Arrival and Departure:** Handle the arrival and departure of flows, updating the utilization vector accordingly.
- **Record Utilization:** Record the utilization vector before and after each flow arrival.
- **Store Results:** Store the results, including the path taken and the utilization before and after each flow.

# Result of MLU algo and Infeasibility in solution.

The flows out of single TM resulted in unfeasible solution. Here even after relaxing the constraint of capacity, solution is still infeasible. A further understanding and of system, constraints and algorithm is required.

```
[11]: MLU_df.iloc[:,1:-1]
```

| [11]: | Unnamed: 0 | source_node | destination_node | TOA  | Duration   | Bandwidth | Utilization_Before_0 | Utilization_Before_1 | Utilization_Before_2 | Utilization_Before_3 |
|-------|------------|-------------|------------------|------|------------|-----------|----------------------|----------------------|----------------------|----------------------|
|       | 0          | 0           | 16.0             | 3.0  | 0.197832   | 65.979554 | 43.729103            | 0.000000             | 0.000000             | 0.000000             |
|       | 1          | 1           | 23.0             | 21.0 | 2.075432   | 69.243544 | 66.926802            | 0.000000             | 0.000000             | 0.000000             |
|       | 2          | 2           | 16.0             | 7.0  | 2.588421   | 71.651359 | 28.382993            | 0.000000             | 0.000000             | 0.000000             |
|       | 3          | 3           | 1.0              | 17.0 | 2.834866   | 54.732502 | 251.057546           | 0.000568             | 0.000000             | 0.000568             |
|       | 4          | 4           | 5.0              | 16.0 | 3.233363   | 75.384519 | 93.174743            | 0.000568             | 0.000000             | 0.005589             |
|       | ...        | ...         | ...              | ...  | ...        | ...       | ...                  | ...                  | ...                  | ...                  |
|       | 951        | 951         | 10.0             | 17.0 | 665.108687 | 65.397093 | 293.879951           | 0.203668             | 0.091776             | 0.279628             |
|       | 952        | 952         | 9.0              | 12.0 | 665.856917 | 63.661599 | 287.334515           | 0.209546             | 0.091776             | 0.285506             |
|       | 953        | 953         | 3.0              | 14.0 | 667.108289 | 45.328557 | 24.739861            | 0.215293             | 0.091776             | 0.285506             |
|       | 954        | 954         | 19.0             | 12.0 | 667.493233 | 55.110349 | 215.016937           | 0.215293             | 0.091776             | 0.285506             |
|       | 955        | 955         | 8.0              | 22.0 | 667.622801 | 57.663126 | 182.300979           | 0.215293             | 0.091776             | 0.289806             |

956 rows x 154 columns

- **Track Active Flows:** Maintain a list of currently active flows.
- **Flow Arrival and Departure:** Handle the arrival and departure of flows, updating the utilization vector accordingly.
- **Record Utilization:** Record the utilization vector before and after each flow arrival.
- **Store Results:** Store the results, including the path taken and the utilization before and after each flow.